

MERN stack - Shopping Online

Lab 04

Contents

1	MERN stack	2
1.1	Functionals	2
1.1.1	Admin - listproduct	2
1.1.2	Admin - addproduct	6
1.1.3	Admin - updateproduct	8
1.1.4	Admin - deleteproduct	10

1 MERN stack

1.1 Functionals

1.1.1 Admin - listproduct

1.1.1.1 Server

- Create 'server/models/ProductDAO.js' file

```
1 require('../utils/MongooseUtil');
2 const Models = require('../Models');
3
4 const ProductDAO = {
5   async selectAll() {
6     const query = {};
7     const products = await Models.Product.find(query).exec();
8     return products;
9   }
10 };
11 module.exports = ProductDAO;
```

- Update 'server/api/admin.js' file

```
1 ...
2 // daos
3 const ProductDAO = require('../models/ProductDAO');
4 ...
5 // product
6 router.get('/products', JwtUtil.checkToken, async function (req, res) {
7   // get data
8   var products = await ProductDAO.selectAll();
9   // pagination
10  const sizePage = 4;
11  const noPages = Math.ceil(products.length / sizePage);
12  var curPage = 1;
13  if (req.query.page) curPage = parseInt(req.query.page); // /products?page=xxx
14  const offset = (curPage - 1) * sizePage;
15  products = products.slice(offset, offset + sizePage);
16  // return
17  const result = { products: products, noPages: noPages, curPage: curPage };
18  res.json(result);
19 });
20 ...
```

- Postman: (GET) <http://localhost:3000/api/admin/products?page=1>

- Headers: "x-access-token": <token>

1.1.1.2 Client-admin

- Update 'client-admin/src/components/MainComponent.js' file

```
1 ...
2 import Product from './ProductComponent';
3
4 class Main extends Component {
5   ...
6   render() {
7     ...
8     return (
9       ...
10      <Routes>
11        ...
12        <Route path='/admin/product' element={<Product />} />
13      </Routes>
14      ...
15    );
16    ...
```

```

17   }
18 }
19 ...

```

- Update 'client-admin/src/components/MenuComponent.js' file

```

1 ...
2 class Menu extends Component {
3   ...
4   render() {
5     return (
6       ...
7       <ul className="menu">
8         ...
9         <li className="menu"><Link to="/admin/product">Product</Link></li>
10        ...
11      </ul>
12      ...
13    );
14  }
15  ...
16 }
17 ...

```

- Create 'client-admin/src/components/ProductComponent.js' file

```

1 import axios from 'axios';
2 import React, { Component } from 'react';
3 import MyContext from '../contexts/MyContext';
4 import ProductDetail from './ProductDetailComponent';
5
6 class Product extends Component {
7   static contextType = MyContext; // using this.context to access global state
8   constructor(props) {
9     super(props);
10    this.state = {
11      products: [],
12      noPages: 0,
13      curPage: 1,
14      itemSelected: null
15    };
16  }
17   render() {
18     const prods = this.state.products.map((item) => {
19       return (
20         <tr key={item._id} className="datatable" onClick={() => this.trItemClick(item)}>
21           <td>{item._id}</td>
22           <td>{item.name}</td>
23           <td>{item.price}</td>
24           <td>{new Date(item.cdate).toLocaleString()}</td>
25           <td>{item.category.name}</td>
26           <td><img src={"data:image/jpg;base64," + item.image} width="100px" height="100px" alt
              ="" /></td>
27         </tr>
28       );
29     });
30     const pagination = Array.from({ length: this.state.noPages }, (_, index) => {
31       if ((index + 1) === this.state.curPage) {
32         return <span key={index}>| <b>{index + 1}</b> |</span>;
33       } else {
34         return <span key={index} className="link" onClick={() => this.lnkPageClick(index + 1)}
              >| {index + 1} |</span>;
35       }
36     });
37     return (
38       <div>
39         <div className="float-left">
40           <h2 className="text-center">PRODUCT LIST</h2>
41           <table className="datatable" border="1">
42             <tbody>

```

```

43         <tr className="datatable">
44             <th>ID</th>
45             <th>Name</th>
46             <th>Price</th>
47             <th>Creation date</th>
48             <th>Category</th>
49             <th>Image</th>
50         </tr>
51         {prods}
52         <tr>
53             <td colSpan="6">{pagination}</td>
54         </tr>
55     </tbody>
56 </table>
57 </div>
58 <div className="inline" />
59 <ProductDetail item={this.state.itemSelected} />
60 <div className="float-clear" />
61 </div>
62 );
63 }
64 componentDidMount() {
65     this.apiGetProducts(this.state.curPage);
66 }
67 // event-handlers
68 lnkPageClick(index) {
69     this.apiGetProducts(index);
70 }
71 trItemClick(item) {
72     this.setState({ itemSelected: item });
73 }
74 // apis
75 apiGetProducts(page) {
76     const config = { headers: { 'x-access-token': this.context.token } };
77     axios.get('/api/admin/products?page=' + page, config).then((res) => {
78         const result = res.data;
79         this.setState({ products: result.products, noPages: result.noPages, curPage: result.
            curPage });
80     });
81 }
82 }
83 export default Product;

```

- Create 'client-admin/src/components/ProductDetailComponent.js' file

```

1  import axios from 'axios';
2  import React, { Component } from 'react';
3  import MyContext from '../contexts/MyContext';
4
5  class ProductDetail extends Component {
6      static contextType = MyContext; // using this.context to access global state
7      constructor(props) {
8          super(props);
9          this.state = {
10             categories: [],
11             txtID: '',
12             txtName: '',
13             txtPrice: 0,
14             cmbCategory: '',
15             imgProduct: '',
16         };
17     }
18     render() {
19         const cates = this.state.categories.map((cate) => {
20             if (this.props.item !== null) {
21                 return <option key={cate._id} value={cate._id} selected={cate._id === this.props.item.
                    category._id}>{cate.name}</option>;
22             } else {
23                 return <option key={cate._id} value={cate._id}>{cate.name}</option>;

```

```

24     }
25   });
26   return (
27     <div className="float-right">
28       <h2 className="text-center">PRODUCT DETAIL</h2>
29       <form>
30         <table>
31           <tbody>
32             <tr>
33               <td>ID</td>
34               <td><input type="text" value={this.state.txtID} onChange={(e) => { this.
35                 setState({ txtID: e.target.value }) }} readOnly={true} /></td>
36             </tr>
37             <tr>
38               <td>Name</td>
39               <td><input type="text" value={this.state.txtName} onChange={(e) => { this.
40                 setState({ txtName: e.target.value }) }} /></td>
41             </tr>
42             <tr>
43               <td>Price</td>
44               <td><input type="text" value={this.state.txtPrice} onChange={(e) => { this.
45                 setState({ txtPrice: e.target.value }) }} /></td>
46             </tr>
47             <tr>
48               <td>Image</td>
49               <td><input type="file" name="fileImage" accept="image/jpeg, image/png, image/
50                 gif" onChange={(e) => this.previewImage(e)} /></td>
51             </tr>
52             <tr>
53               <td>Category</td>
54               <td><select onChange={(e) => { this.setState({ cmbCategory: e.target.value })
55                 }}>{cates}</select></td>
56             </tr>
57             <tr>
58               <td></td>
59               <td>
60                 <input type="submit" value="ADD NEW" />
61                 <input type="submit" value="UPDATE" />
62                 <input type="submit" value="DELETE" />
63               </td>
64             </tr>
65             <tr>
66               <td colspan="2"><img src={this.state.imgProduct} width="300px" height="300px"
67                 alt="" /></td>
68             </tr>
69           </tbody>
70         </table>
71       </form>
72     </div>
73   );
74 }
75 componentDidMount() {
76   this.apiGetCategories();
77 }
78 componentDidUpdate(prevProps) {
79   if (this.props.item !== prevProps.item) {
80     this.setState({
81       txtID: this.props.item._id,
82       txtName: this.props.item.name,
83       txtPrice: this.props.item.price,
84       cmbCategory: this.props.item.category._id,
85       imgProduct: 'data:image/jpg;base64,' + this.props.item.image
86     });
87   }
88 }
89 // event-handlers
90 previewImage(e) {
91   const file = e.target.files[0];
92   if (file) {

```

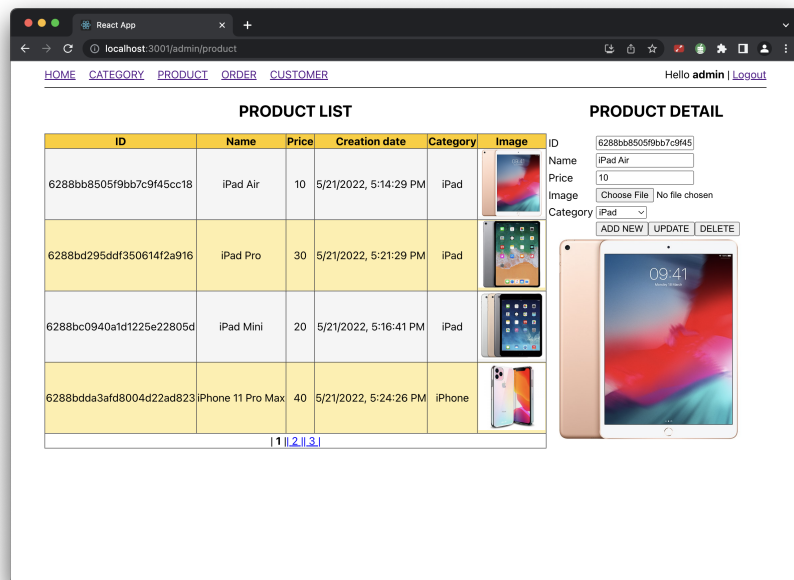
```

87     const reader = new FileReader();
88     reader.onload = (evt) => {
89         this.setState({ imgProduct: evt.target.result });
90     }
91     reader.readAsDataURL(file);
92 }
93 }
94 // apis
95 apiGetCategories() {
96     const config = { headers: { 'x-access-token': this.context.token } };
97     axios.get('/api/admin/categories', config).then((res) => {
98         const result = res.data;
99         this.setState({ categories: result });
100     });
101 }
102 }
103 export default ProductDetail;

```

- Browser: <http://localhost:3001/admin/product>

1.1.1.3 Screenshots



1.1.2 Admin - addproduct

1.1.2.1 Server

- Update 'server/models/CategoryDAO.js' file

```

1 ...
2 const CategoryDAO = {
3     ...,
4     async selectById(_id) {
5         const category = await Models.Category.findById(_id).exec();
6         return category;
7     }
8 };
9 ...

```

- Update 'server/models/ProductDAO.js' file

```

1 ...
2 const ProductDAO = {

```

```

3   ...,
4   async insert(product) {
5       const mongoose = require('mongoose');
6       product._id = new mongoose.Types.ObjectId();
7       const result = await Models.Product.create(product);
8       return result;
9   }
10  };
11  ...

```

- Update 'server/api/admin.js' file

```

1   ...
2   // product
3   router.post('/products', JwtUtil.checkToken, async function (req, res) {
4       const name = req.body.name;
5       const price = req.body.price;
6       const cid = req.body.category;
7       const image = req.body.image;
8       const now = new Date().getTime(); // milliseconds
9       const category = await CategoryDAO.selectByID(cid);
10      const product = { name: name, price: price, image: image, cdate: now, category: category };
11      const result = await ProductDAO.insert(product);
12      res.json(result);
13  });
14  ...

```

- Postman: (POST) http://localhost:3000/api/admin/products

- Headers: "x-access-token": <token>
- Body (raw+JSON): { "name": "iWatch S7", "price": 130, "category": "644279d8146cdb5d63250036", "image": "/9j/4AAQSkZJRgABAQEAAAAD/..." }

1.1.2.2 Client-admin

- Update 'client-admin/src/components/ProductComponent.js' file

```

1   ...
2   class Product extends Component {
3       ...
4       render() {
5           ...
6           return (
7               <div>
8                   ...
9                   <ProductDetail item={this.state.itemSelected} curPage={this.state.curPage}
10                      updateProducts={this.updateProducts} />
11               </div>
12           );
13       }
14       updateProducts = (products, noPages) => { // arrow-function
15           this.setState({ products: products, noPages: noPages });
16       }
17       ...
18   }
19   ...

```

- Update 'client-admin/src/components/ProductDetailComponent.js' file

```

1   ...
2   class ProductDetail extends Component {
3       ...
4       render() {
5           ...
6           return (
7               ...
8               <input type="submit" value="ADD NEW" onClick={(e) => this.btnAddClick(e)} />

```

```

9      ...
10    });
11    ...
12  }
13  // event-handlers
14  btnAddClick(e) {
15    e.preventDefault();
16    const name = this.state.txtName;
17    const price = parseInt(this.state.txtPrice);
18    const category = this.state.cmbCategory;
19    const image = this.state.imgProduct.replace(/^data:image\/[a-z]+;base64/, ''); // remove "
      data:image/...;base64,"
20    if (name && price && category && image) {
21      const prod = { name: name, price: price, category: category, image: image };
22      this.apiPostProduct(prod);
23    } else {
24      alert('Please input name and price and category and image');
25    }
26  }
27  // apis
28  apiPostProduct(prod) {
29    const config = { headers: { 'x-access-token': this.context.token } };
30    axios.post('/api/admin/products', prod, config).then((res) => {
31      const result = res.data;
32      if (result) {
33        alert('OK BABY!');
34        this.apiGetProducts();
35      } else {
36        alert('SORRY BABY!');
37      }
38    });
39  }
40  apiGetProducts() {
41    const config = { headers: { 'x-access-token': this.context.token } };
42    axios.get('/api/admin/products?page=' + this.props.curPage, config).then((res) => {
43      const result = res.data;
44      this.props.updateProducts(result.products, result.noPages);
45    });
46  }
47  ...
48 }
49 ...

```

- Browser: <http://localhost:3001/admin/products>

1.1.3 Admin - updateproduct

1.1.3.1 Server

- Update 'server/models/ProductDAO.js' file

```

1  ...
2  const ProductDAO = {
3    ...,
4    async selectByID(_id) {
5      const product = await Models.Product.findById(_id).exec();
6      return product;
7    },
8    async update(product) {
9      const newvalues = { name: product.name, price: product.price, image: product.image,
      category: product.category };
10     const result = await Models.Product.findByIdAndUpdate(product._id, newvalues, { new: true
      });
11     return result;
12   }
13 };
14 ...

```

- Update 'server/api/admin.js' file

```

1  ...
2  // product
3  router.put('/products', JwtUtil.checkToken, async function (req, res) {
4      const _id = req.body.id;
5      const name = req.body.name;
6      const price = req.body.price;
7      const cid = req.body.category;
8      const image = req.body.image;
9      const now = new Date().getTime(); // milliseconds
10     const category = await CategoryDAO.selectByID(cid);
11     const product = { _id: _id, name: name, price: price, image: image, cdate: now, category:
        category };
12     const result = await ProductDAO.update(product);
13     res.json(result);
14 });
15 ...

```

- Postman: (PUT) `http://localhost:3000/api/admin/products/<id>`

- Headers: "x-access-token": <token>
- Body (raw+JSON): { "name": "iWatch S7", "price": 130, "category": "644279d8146cdb5d63250036", "image": "/9j/4AAQSkZJRgABAQEAAAAAAAAAD/..." }

1.1.3.2 Client-admin

- Update 'client-admin/src/components/ProductDetailComponent.js' file

```

1  ...
2  class ProductDetail extends Component {
3      ...
4      render() {
5          ...
6          return (
7              ...
8              <input type="submit" value="UPDATE" onClick={(e) => this.btnUpdateClick(e)} />
9              ...
10         );
11         ...
12     }
13     // event-handlers
14     btnUpdateClick(e) {
15         e.preventDefault();
16         const id = this.state.txtID;
17         const name = this.state.txtName;
18         const price = parseInt(this.state.txtPrice);
19         const category = this.state.cmbCategory;
20         const image = this.state.imgProduct.replace(/^data:image\/[a-z]+;base64/, ''); // remove "
            data:image/...;base64,"
21         if (id && name && price && category && image) {
22             const prod = { name: name, price: price, category: category, image: image };
23             this.apiPutProduct(id, prod);
24         } else {
25             alert('Please input id and name and price and category and image');
26         }
27     }
28     // apis
29     apiPutProduct(id, prod) {
30         const config = { headers: { 'x-access-token': this.context.token } };
31         axios.put('/api/admin/products/' + id, prod, config).then((res) => {
32             const result = res.data;
33             if (result) {
34                 alert('OK BABY!');
35                 this.apiGetProducts();
36             } else {
37                 alert('SORRY BABY!');
38             }
39         });
40     }
41 }

```

```

39     });
40   }
41   ...
42 }
43 ...

```

- Browser: <http://localhost:3001/admin/product>

1.1.4 Admin - deleteproduct

1.1.4.1 Server

- Update 'server/models/ProductDAO.js' file

```

1 ...
2 const ProductDAO = {
3   ...,
4   async delete(_id) {
5     const result = await Models.Product.findByIdAndRemove(_id);
6     return result;
7   }
8 };
9 ...

```

- Update 'server/api/admin.js' file

```

1 ...
2 // product
3 router.delete('/products/:id', JwtUtil.checkToken, async function (req, res) {
4   const _id = req.params.id;
5   const result = await ProductDAO.delete(_id);
6   res.json(result);
7 });
8 ...

```

- Postman: (DELETE) <http://localhost:3000/api/admin/products/<id>>

- Headers: "x-access-token": <token>

1.1.4.2 Client-admin

- Update 'client-admin/src/components/ProductDetailComponent.js' file

```

1 ...
2 class ProductDetail extends Component {
3   ...
4   render() {
5     ...
6     return (
7       ...
8       <input type="submit" value="DELETE" onClick={(e) => this.btnDeleteClick(e)} />
9       ...
10    );
11    ...
12  }
13  // event-handlers
14  btnDeleteClick(e) {
15    e.preventDefault();
16    if (window.confirm('ARE YOU SURE?')) {
17      const id = this.state.txtID;
18      if (id) {
19        this.apiDeleteProduct(id);
20      } else {
21        alert('Please input id');
22      }
23    }
24  }
25  // apis

```

```

26  apiDeleteProduct(id) {
27      const config = { headers: { 'x-access-token': this.context.token } };
28      axios.delete('/api/admin/products/' + id, config).then((res) => {
29          const result = res.data;
30          if (result) {
31              alert('OK BABY!');
32              this.apiGetProducts();
33          } else {
34              alert('SORRY BABY!');
35          }
36      });
37  }
38  apiGetProducts() {
39      ...
40      axios.get('/api/admin/products?page=' + this.props.curPage, config).then((res) => {
41          ...
42          if (result.products.length !== 0) {
43              this.props.updateProducts(result.products, result.noPages);
44          } else {
45              axios.get('/api/admin/products?page=' + (this.props.curPage - 1), config).then((res) => {
46                  {
47                      const result = res.data;
48                      this.props.updateProducts(result.products, result.noPages);
49                  }
50              });
51          }
52          ...
53      }
54      ...

```

- Browser: <http://localhost:3001/admin/product>